

Digital Logic Design
Project Report
Modified Stack Implementation



Submitted by:

Muhammad Saad Ramzan (25L-0794)

Mahad Sajid Raja (25L-0644)

Submitted to:

Sir Zummar Saad

Submission Date:

8th May, 2026

Department of Electrical Engineering

National University of Computer and Emerging Sciences, Lahore

Table of Contents

- 1 Introduction
- 2 Problem Analysis
- 3 Design Requirements
- 4 Feasibility Analysis
- 5 Possible Solutions
- 6 Preliminary Design
- 7 Design Description
- 8 Software Simulation
- 9 Experimental Results
- 10 Performance Analysis
- 11 Conclusion
- 12 References

1 Introduction

A stack is a fundamental data structure that operates on the Last In First Out (LIFO) principle. Elements are inserted and removed from the same end, called the top of the stack. This means the most recently added element is always the first one to be removed, which mirrors many real-world scenarios such as a pile of plates where you always pick from the top.

In this project, a Modified Stack capable of storing up to 10 integers, each ranging from 0 to 15 (4-bit values), is implemented using digital logic. The modification to the standard stack behavior is that when the stack is full and a new element is pushed, instead of blocking the operation, the oldest element at the bottom is automatically dropped and all remaining elements shift down by one position, making room for the new element at the top. This gives the stack a sliding window behavior.

The design makes use of registers as memory elements, along with combinational logic for control and status detection. The core concepts applied in this project include:

- 10 4-bit registers for storing each integer
- 40 2-to-1 MUXes (4 per register) for selecting what data each register loads
- 1 Up/Down counter for tracking the number of elements currently stored
- 1 Hex keyboard for 4-bit data input
- Binary input probes for ENQ, DEQ, CLK and CLR control signals
- Output probes for displaying register values and FULL and EMPTY status
- AND, OR, NOT gates for control and flag generation logic

All concepts used in this project fall within the scope of the Digital Logic Design course, specifically registers, counters, and combinational circuit design. The circuit is simulated using LogicWorks software

2 Problem Analysis

The problem requires designing a Modified Stack data structure at the hardware level using digital logic components. The stack must:

- Store up to 10 integers, where each integer is a 4-bit value in the range 0 to 15 in decimal.
- Support a Push operation that adds a new integer to the top of the stack.
- Support a Pop operation that retrieves and removes the integer currently at the top of the stack.
- Detect and signal when the stack is empty to prevent an invalid pop operation.
- Detect and signal when the stack is full.
- Implement the modified behavior: when the stack is full and a Push is requested, the oldest element at the bottom is dropped, all elements shift down by one register, and the new element is loaded into the topmost register. The count remains at 10.
- Provide a clear interface allowing the user to interact with the stack through Push, Pop, Full, and Empty signals.

Each storage slot uses one 4-bit register with four 2-to-1 MUXes controlling its inputs, giving 10 registers and 40 MUXes in total across the stack. A hex keyboard allows the user to enter the desired 4-bit value and binary probes control the ENQ and DEQ operations.

3 Design Requirements

Input:

- A 4-bit binary integer representing values 0 to 15 in decimal to be pushed onto the stack.
- An ENQ (Push) control signal to trigger the push operation.
- A DEQ (Pop) control signal to trigger the pop operation.

Output:

- A 4-bit binary output representing the integer at the top of the stack.
- A FULL flag signal that is logic HIGH when the stack contains 10 elements.
- An EMPTY flag signal that is logic HIGH when the stack contains 0 elements.

Constraints:

- Stack capacity is exactly 10 integers.
- Each integer must be in the range 0 to 15, stored as a 4-bit binary value.
- Normal Push operation is only allowed when the stack is not full.
- Pop operation is blocked when the EMPTY flag is HIGH.
- LIFO ordering must be strictly maintained at all times.
- When the stack is full and a Push is requested, the shift-overwrite behavior must activate automatically without any additional user input.
- The design must be implementable and simulated in LogicWorks.

4 Feasibility Analysis

Time Management:

The design was divided into three phases. The first phase involved designing individual components including the 4-bit MUX-register blocks, the counter for tracking occupancy, and the Full and Empty detection logic. The second phase covered integration and wiring of all ten register blocks along with the control logic in LogicWorks. The third phase was dedicated to testing, simulation, and verification of correct LIFO and shift-overwrite behavior. This phased approach was completed within the project deadline.

Cost Management:

Since the entire project is implemented as a software simulation in LogicWorks, there is no physical hardware cost involved. The only resource required is access to a computer with LogicWorks installed, which is available in the university computer labs. This makes the project fully cost-feasible.

Technical Feasibility:

All components used are covered in the course: Registers, 2-to-1 MUXes, Up/Down counter, and basic AND, OR, NOT gates for combinational control logic.

5 Possible Solutions

Two main approaches were considered for implementing the Modified Stack:

Solution 1: Pointer-Based Stack

In this approach, a single pointer tracks the current top of the stack. Push increments the pointer and stores data at that location. Pop reads from the pointer location and decrements it. However, implementing the modified full behavior in this approach becomes complex because when the stack is full and a push is requested, the pointer-based approach requires additional logic to identify and invalidate the oldest element while reusing that register slot. Managing this with a circular pointer while maintaining strict LIFO order adds significant complexity to the control logic.

Solution 2: Shift Register-Based Stack (Chosen)

In this approach, the registers are arranged in a fixed linear order. Register 10 always holds the most recently pushed element (top of stack). Register 1 always holds the oldest element (bottom of stack). On a normal Push, the new element goes into the next available register from the bottom up. On a Pop, the topmost occupied register is read and cleared. When the stack is full and a new Push occurs, a Shift signal is activated. All registers shift down by one position (Register N loads from Register N+1), Register 1 is overwritten and effectively dropped, and the new element is loaded into Register 10. This approach maps cleanly onto the hardware and makes the shift-overwrite behavior simple to implement using MUX logic already present in each register block.

Chosen Solution: Solution 2 (Shift Register-Based Stack)

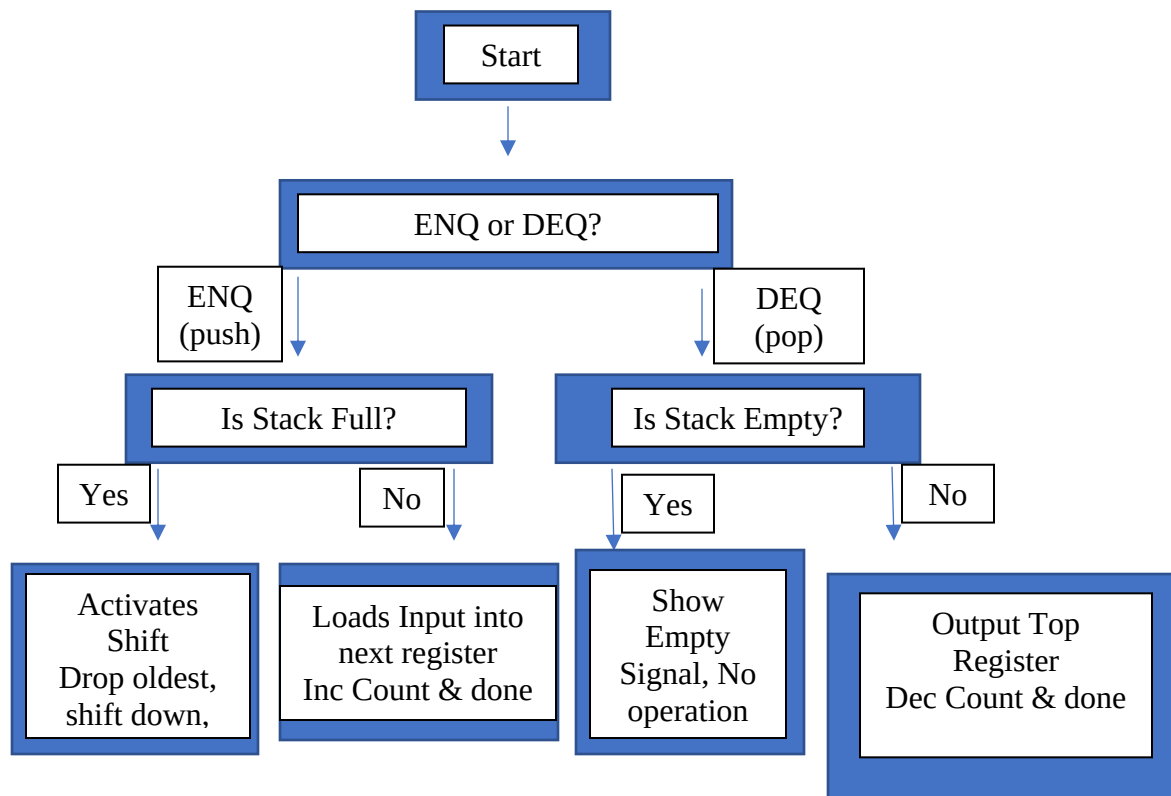
Solution 2 is chosen because the shift behavior required by the Modified Stack maps directly and naturally onto a fixed linear register arrangement. The MUX-based selector already present in each register block can accommodate the shift path with minimal additions. The control logic remains simple and easy to verify in simulation.

6 Preliminary Design

The system consists of the following main blocks:

- **Data Registers:** Ten 4-bit registers (MUX and Register 1 through 10), each holding one stored integer. Register 10 is the top of the stack and Register 1 is the bottom.
- **Element Counter:** A counter tracking the number of elements currently stored, used to generate the Full and Empty flags.
- **Full and Empty Detection Logic:** Combinational logic that outputs the FULL flag when count equals 10 and the EMPTY flag when count equals 0.
- **Shift Signal Generator:** A single AND gate that produces the SHIFT signal when both FULL and ENQ are simultaneously active.
- **MUX Control Logic:** Per-register combinational logic that selects what each register loads on the next clock edge — new input, shifted data from the register above, or its own current value.

Flowchart of Queue Operation:



7 Design Description

The design is composed of the following hardware blocks:

4-bit MUX and Register Blocks (MUX and Register 1 to 10):

Each of the ten storage locations consists of a DEV2 4-bit register with four 2-to-1 MUXes controlling its four input bits. AND and OR gates handle control signal routing into each MUX select line. For each bit position, two AND gates and one OR gate form a 2-to-1 selector that controls what value is loaded into the register on the next rising clock edge. One AND gate passes the new input data when a normal push is active. The other AND gate passes the output of the register directly above it when the SHIFT signal is active. The OR gate combines both paths and feeds into the D input of the register. This structure allows each register to either accept new data or shift data from its neighbor, depending on the control signals.

Element Counter:

A synchronous counter tracks the number of elements currently stored in the stack. It increments by 1 on each successful normal Push and decrements by 1 on each successful Pop. During a shift-overwrite Push (when FULL and ENQ are both active), the counter remains unchanged at 10 since no net change in occupancy occurs.

Full and Empty Detection Logic:

The FULL flag is generated by combinational logic that outputs HIGH when the counter value equals 10. The EMPTY flag is generated by combinational logic that outputs HIGH when the counter value equals 0. These flags are used both as status indicators for the user and as inputs to the control logic.

Shift Signal Generator:

A single 2-input AND gate takes the FULL flag and the ENQ signal as inputs and produces the SHIFT signal. When both are HIGH simultaneously, SHIFT goes HIGH, enabling the shift-overwrite behavior across all ten register blocks in the same clock cycle.

ENQ Masking Logic:

To prevent the normal push path from activating during a shift-overwrite operation, the ENQ signal is masked using an AND gate with NOT FULL. This masked ENQ signal (ENQ AND NOT FULL) is used as the enable for the new-input AND gates in each register block, ensuring that during a shift, only the shift path is active and not the normal load path.

Push and Pop Control:

The RunClk signal controls whether the clock is passed to the registers and counter. It is generated from the control logic such that the clock runs on a valid Push (ENQ and NOT FULL), a valid Pop (DEQ and NOT EMPTY), or a shift-overwrite Push (SHIFT active). This prevents spurious clock edges from corrupting register contents.

Output:

The top of the stack output is always taken from the register corresponding to the current topmost occupied slot, selected based on the counter value. The 4-bit output is displayed using output probes in LogicWorks.

8 Software Simulation

The design is implemented and simulated using LogicWorks software. LogicWorks is a digital circuit simulation environment that supports placement of logic gates, registers, counters, and other standard components to construct and verify digital circuits.

Simulation Setup:

- Software Used: LogicWorks
- Clock: A single global clock drives all DEV2 registers and the element counter synchronously.
- Input: A Hex keyboard provides the 4-bit data input. Binary input probes control the ENQ, DEQ, CLK and CLR signals.
- Output: Output probes display the current contents of each register and two indicator probes show the FULL and EMPTY status flags.
- The complete circuit file is submitted separately as a cct LogicWorks file.

Simulation Steps:

Step 1: The LogicWorks schematic is constructed with all ten MUX and Register blocks, the element counter, Full and Empty detection logic, the SHIFT signal generator, and the ENQ masking gate wired together as described in the Design Description.

Step 2: The circuit is verified in its initial state. All registers hold 0, the EMPTY flag is HIGH, and the FULL flag is LOW.

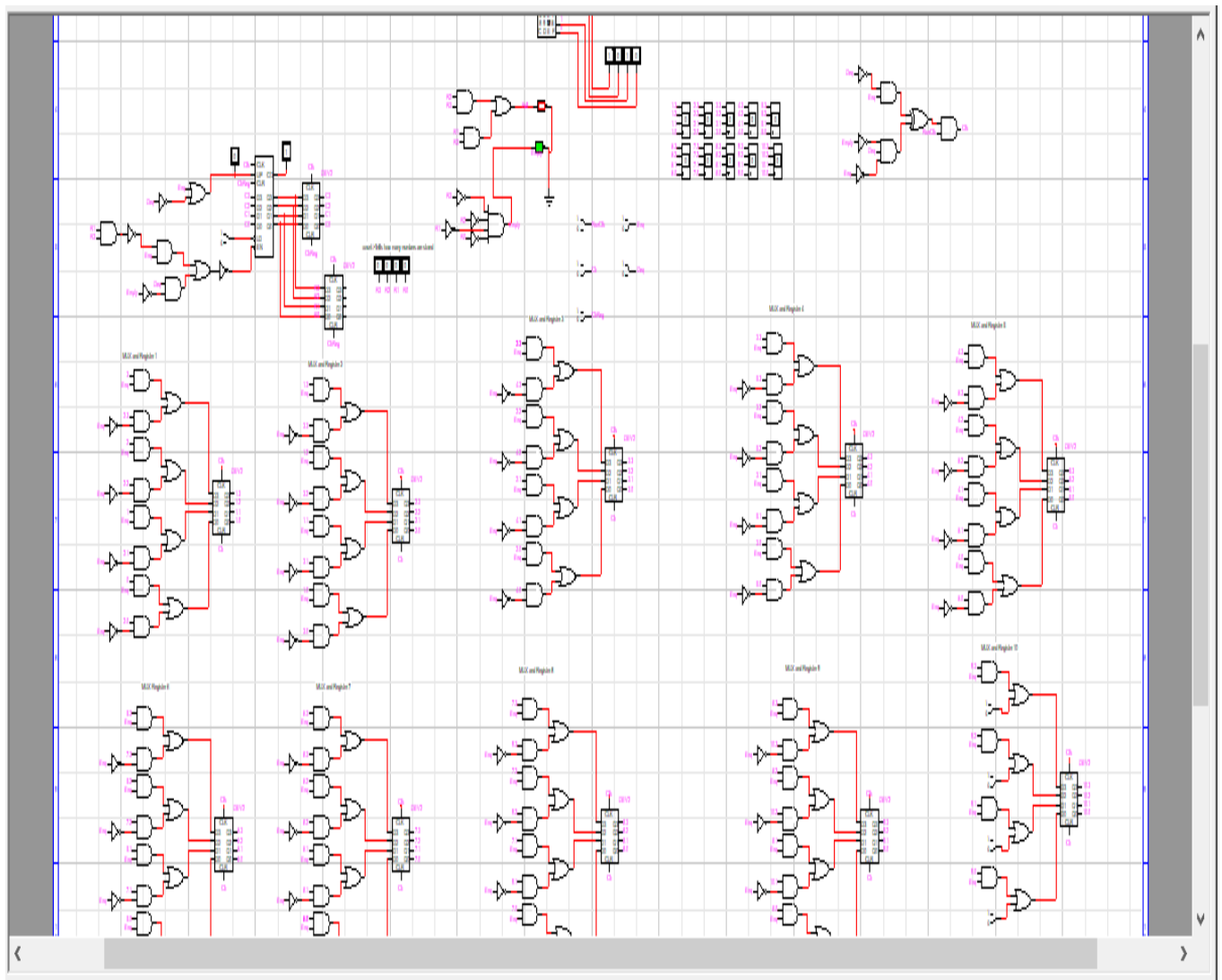
Step 3: Elements are pushed one by one by setting the 4-bit input and asserting ENQ. The counter increments after each push and the correct register loads the input value.

Step 4: Push is attempted when the stack is full with a new value. The SHIFT signal activates, all registers shift down by one, register 1 content is dropped, and the new value appears at Register 10. The counter remains at 10.

Step 5: Elements are popped one by one by asserting DEQ. Each pop retrieves the most recently pushed element, confirming correct LIFO ordering.

Step 6: Pop is attempted on an empty stack. The EMPTY flag remains HIGH and no operation occurs, confirming underflow protection.

Screenshot of the Circuit implemented on LOGIC WORKS:



9 Experimental Results

The following results were observed during LogicWorks simulation. The Modified Stack was tested across four scenarios to verify all required behaviors.

Test 1: Sequential Push and Pop (LIFO Verification)

Four values were pushed onto the stack in order: 2 (0010), 6 (0110), 9 (1001), and 13 (1101).

The pop operation was then applied four times. The output produced the values in reverse order: 13, 9, 6, 2. This confirms correct LIFO behavior.

Test 2: Empty Stack Detection

Starting from an empty stack, a pop operation was attempted. The EMPTY flag was HIGH and no removal was performed, confirming underflow protection works correctly.

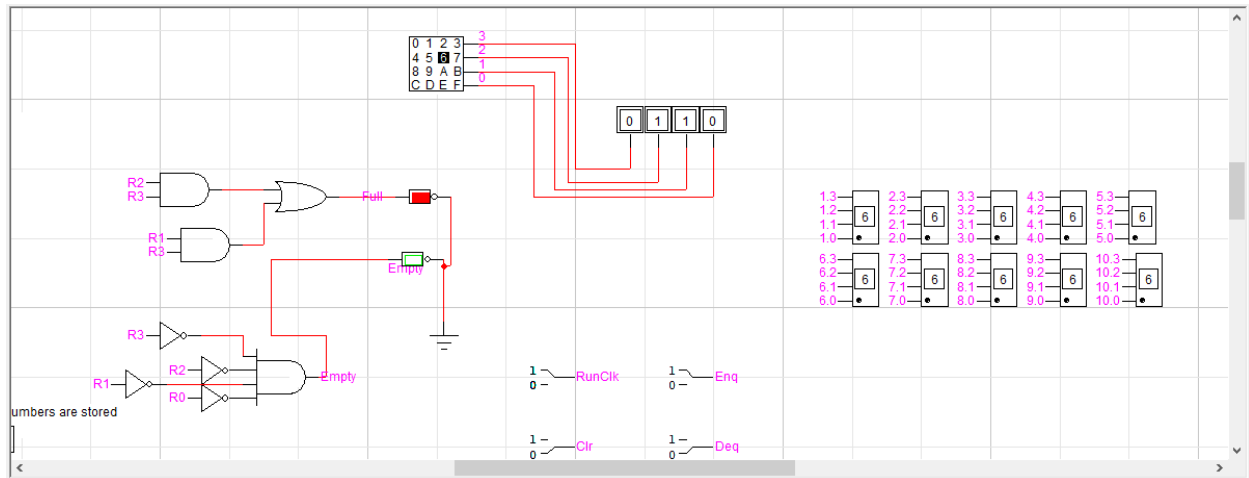
Test 3: Full Stack with Normal Push Blocked

Ten elements were pushed sequentially. After the tenth push, the FULL flag went HIGH. A normal push was attempted without triggering the shift path. The FULL flag remained HIGH and no data was loaded, confirming that the ENQ masking logic correctly blocks normal pushes when full.

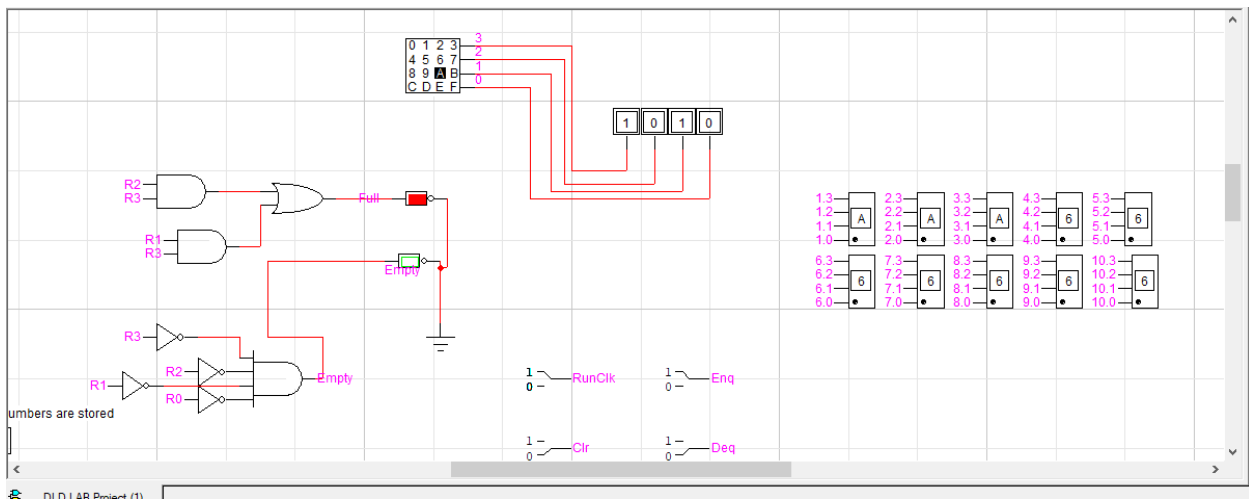
Test 4: Modified Behavior (Shift Overwrite)

With the stack full at 10 elements, a new value was pushed. The SHIFT signal activated. All register contents shifted down by one position, the value previously in Register 1 was dropped, and the new value was loaded into Register 10. The FULL flag remained HIGH and the counter stayed at 10. Popping elements afterward confirmed that the new value was correctly at the top and the order of remaining elements was preserved. This confirms the core modified behavior works correctly.

When the stack is full:



When we take another input, according to the modified stack circuit requirements, the latest one will be over written and pushed back following the LIFO structure while the counter still says 10.



10 Performance Analysis

The implemented Modified Stack design performs correctly across all test cases. The following observations are made regarding its performance:

Correctness: The stack strictly maintains LIFO ordering in all test scenarios. The FULL and EMPTY flags respond correctly. The shift-overwrite behavior activates reliably whenever the stack is full and a push is requested, with no data corruption observed in any register.

Speed: Since the design is fully synchronous, all operations including normal push, pop, and shift-overwrite complete in a single clock cycle. There is no multi-cycle penalty for the shift operation because all ten registers shift simultaneously in parallel on the same clock edge, driven by the same SHIFT signal.

Resource Usage: The design uses 10 4-bit registers and 40 2-to-1 MUXes across all register blocks. A single Up/Down counter tracks stack occupancy. Additional AND, OR and NOT gates handle SHIFT signal generation, ENQ masking, and Full and Empty detection. User interaction is handled through a Hex keyboard for input and binary probes for control signals.

Reliability: The synchronous design eliminates race conditions. The single global clock ensures all registers update simultaneously, which is essential for the shift operation to work correctly. The ENQ masking logic ensures that the normal push path and the shift path never activate at the same time, preventing any possibility of data corruption.

11 Conclusion

This project successfully demonstrates the implementation of a Modified Stack at the digital logic level using concepts covered in the Digital Logic Design course. The stack stores up to 10 integers in the range 0 to 15, each represented as a 4-bit binary value, and operates on the LIFO principle with an additional shift-overwrite behavior when full.

The design uses Registers as storage elements, MUX-based combinational logic for data path selection in each register block, a synchronous counter for occupancy tracking, and simple AND and OR gate logic for Full and Empty detection and Shift signal generation. All ten MUX and Register blocks are integrated into a complete, functional system simulated in LogicWorks.

Simulation results confirm that the stack correctly pushes and pops elements in strict LIFO order, generates accurate FULL and EMPTY status signals, handles underflow protection, and most importantly executes the modified shift-overwrite behavior correctly when the stack is full and a new push is requested. The shift-register based approach proved to be the right choice for this design as it maps the modified behavior directly onto hardware with minimal control complexity.

The project reinforces key digital design concepts including sequential circuit design, register-based storage, counter design, and combinational control logic, and extends them into a practical custom data structure implementation.\

References

- 1 M. Morris Mano and Charles R. Kime, Logic and Computer Design Fundamentals, 5th Edition Updated, Prentice Hall.
- 2 Thomas L. Floyd, Digital Fundamentals, 7th Edition, Prentice Hall, 2000.
- 3 John F. Wakerly, Digital Design: Principles and Practices, 5th Edition, Pearson Education, 2001.
- 4 Logic Works Software Documentation, Capilano Computing Systems.